

CiT Backend Engineering Track

Week 8: Phase 1 Capstone — Full-Stack Integration

Finishing, Connecting & Presenting Your Backend API

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 1 — Foundation (Weeks 1–8)
Week	Week 8
Instructor	Jim & Anderson
Duration	6 months 24 weeks
Tech Stack	Java 25, Spring Boot 4, PostgreSQL, OpenAPI, Postman, Git

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Today we complete the foundation You have helped us lay over these eight weeks. We thank You for every concept understood, every error debugged, and every small victory along the way. As we bring the many pieces of our learning together into one finished work, give us focus to complete it well and humility to keep growing beyond it.

As Philippians 1:6 reminds us, we are "confident of this very thing, that He who has begun a good work in you will complete it." Lord, You began this good work in us; carry it on, and let what we have built be a foundation for all that is still to come.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,

Amen.

Scripture for the Week

"Being confident of this very thing, that He who has begun a good work in you will complete it until the day of Jesus Christ."

— Philippians 1:6 (NKJV)

Table of Contents

1. Welcome to Week 8
2. The Capstone Project — What You Are Building
3. Finalising Your REST API
4. CORS — Letting the Browser Talk to Your API
5. Validation & Global Error Handling
6. Designing & Documenting Your API — OpenAPI & Postman
7. Integration — Working with the Frontend
8. Presenting & Deploying Your Work
9. Key Takeaways, Submission & What Comes Next

1. Welcome to Week 8

This is it — the final week of Phase 1. Eight weeks ago, many of you had never written a line of code. Look at what you can do now: you write Java, design with objects, store data in a database, and expose it to the world through a REST API. This week you bring it all together into one finished, working product.

A capstone is a project that proves you can combine everything you have learned into something real. This week is less about new syntax and more about integration, polish, and craftsmanship — turning the pieces you have built into a service that a real frontend can use.

And there is something special: this is a shared capstone. The Frontend track is building a user interface that talks to the very API you finish this week. For the first time, the two halves of the class become one application. This is exactly how software teams work in the real world.

What to Expect in Week 8

By the end of this session, you will be able to:

- Describe the shared capstone product and your part in it
- Finalise a clean, complete REST API for the Student Grade Portal
- Structure the service in clean layers: controller → facade → service → repository
- Enable CORS so a browser-based frontend can call your API
- Validate incoming data and return clear, consistent error responses
- Design your API with an OpenAPI (OAS 3.1) specification and share it with the frontend team
- Test every endpoint with Postman
- Present your work and understand your path into Phase 2

2. The Capstone Project — What You Are Building

The whole class builds one product together: the CiT Student Grade Portal. It is a small but complete full-stack application for managing students and their grades — the natural finish of the Student Grade Management System you have grown since Week 5.

2.1 The Two Halves

A full-stack application has two parts that talk over HTTP. Understanding both — and where your responsibility ends and the other begins — is the whole point of this week.

Backend (your work)	Frontend (their work)
A Spring Boot REST API	An HTML/CSS/JavaScript web page
Stores students in PostgreSQL	Shows students in the browser
Serves JSON at /api/students	Fetches that JSON and renders it
Runs on http://localhost:8080	Runs on http://localhost:5500

2.2 The API Contract

The two halves agree on an API contract — the exact endpoints, methods, and JSON shapes both sides will use. This is the single most important artefact of a full-stack team: get it right and the halves fit together perfectly.

Endpoint	Purpose
GET /api/students	Return all students as a JSON array
GET /api/students/{id}	Return one student by id
POST /api/students	Create a student from a JSON body; return the created student
PUT /api/students/{id}	Update an existing student
DELETE /api/students/{id}	Delete a student

This contract is not just a conversation — you will write it down formally as an **OpenAPI specification** (Section 6). That single file becomes the agreement you hand to the frontend team, so both halves can build in parallel against the same source of truth.

Your Capstone Deliverable

A running Spring Boot API that fulfils the contract above, backed by PostgreSQL, with a clean layered structure, validation, error handling, an OpenAPI spec, a README, and a Postman collection — all pushed to GitHub.

3. Finalising Your REST API

Your API works, but "works on my machine" is not the finish line. This section is about turning working code into clean, professional code — the difference between a student project and something a team would trust.

3.1 A Clean Layered Structure

Professional Spring Boot services do not throw all their code into one folder. They separate responsibilities into layers, where each layer has exactly one job. This is the structure used on real engineering teams — including the XENO platform this programme prepares you for — so we will adopt it exactly.

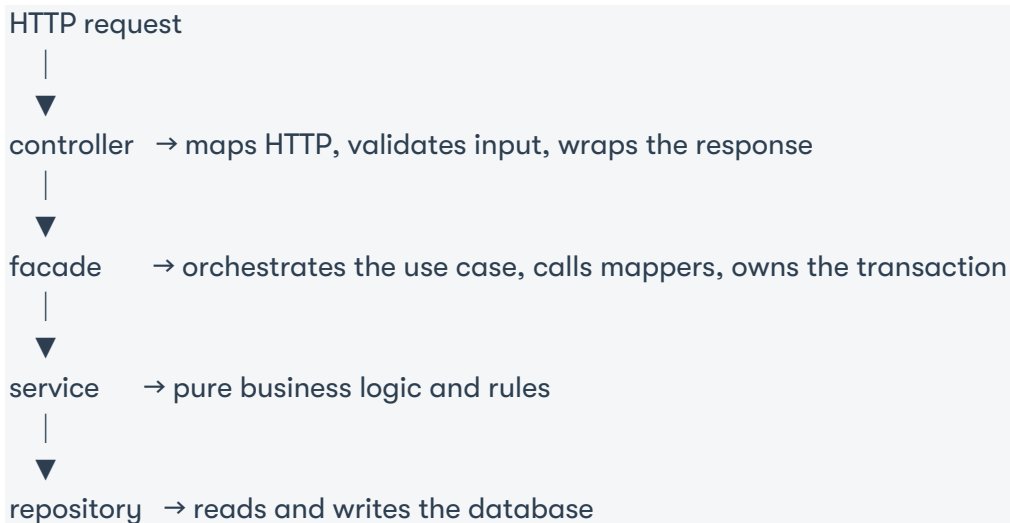
The organising rule is simple: package-by-layer. Every class lives in a package named after its job. Open any well-built service and you instantly know where a controller, a mapper, or a repository lives — without searching.

Package	What Lives There
configuration	Setup classes — security, CORS, and app configuration.
controller	The web layer ONLY. Maps HTTP, validates input, wraps the response. No business logic.
facade	Use-case orchestration. Coordinates services and mappers and owns the transaction for one endpoint.
service	Pure business logic and rules. No web or HTTP awareness — reusable anywhere.
repository	Data access via Spring Data JPA. Reached only from services or facades.
model	The @Entity classes mapped to database tables (enums go in model/enums).
dto	Request/response records — the exact shape of your API surface.
mapper	Converts between entities (model) and DTOs. One mapper per resource.
exception	Custom exceptions and the @RestControllerAdvice handler.
util	Stateless, cross-cutting helpers only — no business logic.

Use these exact, singular package names — controller, service, not variants like controllers or services. Consistency across every service is the whole point.

3.2 The Layered Flow — controller → facade → service → repository

A request flows strictly downward through the layers, and the response flows back up. Each layer only ever talks to the one directly below it.



Layer	Its One Job
Controller	The web layer only. Maps HTTP to and from DTOs, applies validation, returns the response. Contains NO business logic and NEVER calls a repository directly.
Facade	Owns the use case. Orchestrates one or more services, invokes mappers to convert DTOs and entities, and defines the transaction boundary. This is the layer that knows "how to fulfil this endpoint".
Service	Pure, reusable business logic with no web or DTO awareness. The same service can be called by a facade, a scheduled job, or a message listener alike.
Repository	Data access only, using Spring Data JPA. Reached from services or facades.

Why a Facade Layer?

Without a facade, one of two bad things happens: controllers swell with orchestration and mapping logic ("fat controllers"), or services grow web and DTO concerns and stop being reusable. The facade gives orchestration a home of its

own. Controllers stay thin and HTTP-focused, services stay pure and reusable, and the same use case can be driven from an HTTP request, a schedule, or an event without duplicating logic.

3.3 DTOs and Mappers — Protecting Your API Surface

A DTO (Data Transfer Object) is a small class describing exactly the data you send out or accept in — which may differ from your database entity. You would never send a user's password back in JSON; a DTO lets you expose only the safe, relevant fields. It also means changing your database does not accidentally change your public API.

```
// dto package — the API surface
public record StudentResponse(Long id, String name, String regNumber, double gpa) {}
```

Notice this is a record — the concise data carrier from Week 5. Records are ideal for DTOs.

Converting between an entity and a DTO is the job of the mapper package — a dedicated injectable class, one per resource (e.g. StudentMapper). Never put a static from() method on the DTO itself: that would couple your API surface to your database model and reverse the dependency you are trying to protect.

3.4 The Repository Layout

The folders on disk follow the same discipline. Documentation lives in a docs/ directory at the root — never scattered in the project root or inside resources.

```
cit-student-service/
├── build.gradle
├── docs/                # ALL documentation
│   ├── openapi.yaml    # the API spec (Section 6)
│   ├── README.md
│   └── student-portal.postman_collection.json
├── src/
│   ├── main/
│   │   ├── java/com/cit/student/ # the packages from 3.1
│   │   └── resources/
│   │       └── application.yml
```

Finalisation Checklist

- Classes live in the correct package (controller, facade, service, repository, ...)
- Controllers contain no business logic and never touch a repository directly
- The API surface uses DTOs; entities are never returned directly
- Documentation (OpenAPI, README, Postman) lives in docs/, not the project root
- No leftover debug printlns or commented-out code

4. CORS — Letting the Browser Talk to Your API

Here is a problem you will hit the moment the frontend tries to call your API: the browser blocks the request with a "CORS error". This is not a bug — it is a security feature, and you must understand it to build full-stack apps.

4.1 What CORS Is

CORS stands for Cross-Origin Resource Sharing. An "origin" is the combination of protocol, domain, and port — so `http://localhost:5500` (the frontend) and `http://localhost:8080` (your API) are different origins.

By default, browsers block a web page on one origin from calling an API on a different origin. This protects users from malicious sites secretly calling other services. To allow the frontend in, your API must explicitly say "I trust that origin". That permission is CORS.

4.2 Enabling CORS in Spring Boot

The quickest way is the `@CrossOrigin` annotation on your controller, naming the frontend's origin.

```
@CrossOrigin(origins = "http://localhost:5500")
@RestController
@RequestMapping("/api/students")
public class StudentController {
    // ... your endpoints
}
```

4.3 A Global CORS Configuration

For a whole application, configure CORS in one place instead of annotating every controller:

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:5500")
            .allowedMethods("GET", "POST", "PUT", "DELETE");
    }
}
```

```
}  
}
```

When the Frontend Says "CORS Error"

If your teammate reports a CORS error in their browser console, it is almost always the backend's job to fix — the API has not granted their origin permission. Add their exact origin (protocol, domain, and port) to your CORS configuration. This is one of the most common real-world integration conversations between frontend and backend engineers.

5. Validation & Global Error Handling

A public API must defend itself against bad input and fail gracefully. Never trust what a client sends — validate it, and when something goes wrong, return a clear message instead of a stack trace.

5.1 Bean Validation

Spring can validate incoming data automatically using annotations on your fields. Add the rules, then mark the request body with `@Valid`.

```
public class StudentRequest {  
  
    @NotBlank(message = "Name is required")  
    private String name;  
  
    @NotBlank(message = "Registration number is required")  
    private String regNumber;  
  
    @Min(0) @Max(4)  
    private double gpa;  
}
```

```
@PostMapping  
public Student create(@Valid @RequestBody StudentRequest request) {  
    return service.save(request);  
}
```

Annotation	Rule
@NotBlank	Text must be present and not just spaces
@NotNull	The value must not be null
@Min / @Max	A number must fall within a range
@Email	Must be a valid email format
@Size(min, max)	Text or list length must be within bounds

5.2 Global Error Handling

Catch errors across the whole API in one place with `@RestControllerAdvice`. This returns a clean JSON error and the right HTTP status instead of leaking an ugly stack trace.

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<String> handleNotFound(StudentNotFoundException e) {
        return ResponseEntity.status(404).body(e.getMessage());
    }
}
```

Return the Right Status Code

A good API speaks in status codes. Invalid input → 400 Bad Request. Missing resource → 404 Not Found. Success creating → 201 Created. The frontend relies on these to decide what to show the user, so getting them right is part of the contract.

6. Designing & Documenting Your API — OpenAPI & Postman

An API that only you understand is not finished. Professionals describe an API precisely so that others — especially the frontend team — can build against it with confidence. You will use two tools for two different jobs: OpenAPI to design and document the contract, and Postman to test it.

6.1 What Is OpenAPI?

OpenAPI (often called OAS, the OpenAPI Specification, currently version 3.1) is a standard, language-neutral way to describe an entire REST API in a single file — every endpoint, every input, every output, and every data shape. The file is usually written in YAML and named `openapi.yaml`.

The key idea is design-first. Rather than discovering your API by reading the Java code, you write down what the API promises in the spec — and that written promise becomes the contract everyone relies on. Think of it as the architect's blueprint for your API: agreed before and during building, not scribbled afterwards.

Because OpenAPI is a recognised standard, tools understand it. From one `openapi.yaml` you can generate interactive documentation, mock servers, client code, and Postman collections. Write it once; get value everywhere.

6.2 The Anatomy of an OpenAPI Document

Every OpenAPI file has the same top-level sections. Learn these five and you can read or write any spec.

Section	What It Describes
<code>openapi</code>	The spec version you are using, e.g. 3.1.0.
<code>info</code>	The API's title, version, and description.
<code>servers</code>	The base URLs where the API lives (local, staging, production).
<code>paths</code>	Every endpoint — each URL, its HTTP methods, inputs, and responses.
<code>components</code>	Reusable pieces, especially schemas — the shape of your data objects.

6.3 A Worked Example — the Students API

Here is a real OpenAPI 3.1 spec for two endpoints of the Student Grade Portal. First the opening and the paths:

```
openapi: 3.1.0
info:
  title: CiT Student Grade Portal API
  version: 1.0.0
  description: Manage students and their grades.
servers:
  - url: http://localhost:8080
    description: Local development
paths:
  /api/students:
    get:
      summary: List all students
      responses:
        "200":
          description: A list of students
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: "#/components/schemas/Student"
    post:
      summary: Create a student
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/NewStudent"
      responses:
        "201":
          description: The created student
```

Then the reusable data shapes under components/schemas — defined once and referenced everywhere with \$ref:

```
components:
  schemas:
    Student:
```



```
type: object
properties:
  id:    { type: integer }
  name:  { type: string }
  regNumber: { type: string }
  gpa:    { type: number }
NewStudent:
  type: object
  required: [name, regNumber, gpa]
  properties:
    name:    { type: string }
    regNumber: { type: string }
    gpa:      { type: number, minimum: 0, maximum: 4 }
```

Notice how the spec captures everything the frontend needs: the exact field names (regNumber, not reg_number), the types, which fields are required, and even the valid GPA range. No guessing, no mismatches.

6.4 Where the Spec Lives & Sharing It With the Frontend

Keep the spec at docs/openapi.yaml in your repository — committed alongside your code, not left only in a tool. Because it is a standard file, the frontend team can open it in Swagger UI or the Swagger Editor to read interactive documentation, and even import it into their tools.

This file is the single artefact you hand the frontend team. It is your side of the API contract from Section 2. Because it exists as a written agreement, they can start building their portal against it before your code is even finished — both halves work in parallel, meeting in the middle.

The Spec Is the Contract — Keep It in Sync

An API change without a matching spec update is an **incomplete change**. If you add an endpoint or rename a field, update docs/openapi.yaml in the same commit and tell the frontend team. The moment the spec and the real API disagree, the contract is broken and integration bugs appear. Treat the spec with the same seriousness as the code.

6.5 Testing With Postman

Where OpenAPI describes the API, Postman exercises it — actually firing requests so you can confirm it behaves. Handily, you can import your openapi.yaml straight into Postman to generate a ready-made collection, then run the full lifecycle and check the data behaves.

Test	Request	Expect
Create	POST /api/students	201 + the new student
List	GET /api/students	200 + array incl. new one
Get one	GET /api/students/1	200 + that student
Update	PUT /api/students/1	200 + updated student
Delete	DELETE /api/students/1	204, then GET is 404
Bad input	POST with empty name	400 Bad Request

Commit the collection to docs/ too, so any teammate can re-run your tests. Keep it in sync with the spec and the code.

6.6 A README

Finally, every repository needs a README.md in docs/ — the front door of the project. It should let anyone run your API and find the spec. Include:

- A one-paragraph description of what the API does
- Prerequisites (Java 25, PostgreSQL) and how to set up the database
- How to run it (the command and the port)
- A link to the OpenAPI spec at docs/openapi.yaml as the source of truth for endpoints

Exercise: Extend the Spec & Test the Unhappy Path

Two parts:

1. Add the DELETE /api/students/{id} path to the OpenAPI example above — include a path parameter for the id and a "204" response with no body.
2. In Postman, deliberately test failure: send a student with no name, a GPA of 9, and a GET for an id that does not exist. Confirm each returns a sensible status code and message — not a crash.

7. Integration — Working with the Frontend

This is the moment the two tracks meet. Integration is where full-stack projects succeed or stall, and it is as much about communication as about code.

7.1 Agree on the Contract First

Before writing a line, both teams confirm the API contract — and your `docs/openapi.yaml` from Section 6 is that contract. It states the exact URLs, methods, and the JSON field names and types. A single mismatch — `regNumber` versus `reg_number` — will break the connection, which is exactly why the spec exists: to remove the guesswork. Share it early.

7.2 The Common Integration Problems

Symptom (frontend sees)	Usual Cause (backend fixes)
"CORS error" in the console	The frontend origin is not allowed — update CORS config
Fields are undefined	JSON field names do not match what was agreed
404 on a valid-looking URL	Endpoint path or method differs from the contract
400 when creating	Validation rules stricter than the frontend expects
415 Unsupported Media Type	Frontend did not send the JSON Content-Type header

7.3 How to Collaborate Well

You practised these engineering habits in Week 2; now you apply them to a real integration.

- Share your OpenAPI spec, your running API URL, and a Postman collection so the frontend can see exactly what you return
- When something breaks, look at the actual request and response together — do not guess
- Communicate changes: if you rename a field, tell the frontend before you push
- Be generous and patient — you are one team building one product

Integration Is a Team Sport

The hardest bugs in full-stack work live in the gap between two systems, where each team assumes the other is right. The fix is almost always communication: look at the real data crossing the wire, and talk to each other. This soft skill will matter as much as any line of code in your career.

8. Presenting & Deploying Your Work

You built it — now show it. Being able to demonstrate and talk about your work is as important as writing it. This section prepares you to present the capstone with confidence.

8.1 The Demo

You will present a short live demo. Keep it clear and calm — you know this system better than anyone in the room.

3. Start your API and confirm it is running
4. Show the endpoints working in Postman — create, list, update, delete
5. Show the data actually stored in PostgreSQL
6. Then show the frontend calling your live API — the full loop

8.2 Talking About Your Work

Be ready to answer, in plain language: What does it do? How is it structured? What was the hardest part and how did you solve it? What would you improve with more time? Honest reflection impresses more than pretending everything was easy.

8.3 A Note on Deployment

So far your API runs on localhost — your own machine. Deployment means putting it on a server the public can reach. You will go deep on this in Phase 2, but know the shape of it: the app is packaged, a cloud service runs it, and a real database and web address replace your local ones. For the capstone, a clean, well-documented project that runs locally is the goal.

Ship It to GitHub

Whatever else you do, get your capstone onto GitHub with a clear README and a sensible commit history. A well-kept repository is the single most valuable thing you can show a future employer or teammate — it is the proof of everything you learned in Phase 1.

9. Key Takeaways, Submission & What Comes Next

9.1 What You Learned This Week

- How the two halves of a full-stack application fit together through an API contract
- Structuring a service in clean layers — controller → facade → service → repository — with package-by-layer, DTOs, and mappers
- CORS — why browsers block cross-origin calls and how to allow the frontend in
- Validating input with `@Valid` and handling errors globally with `@RestControllerAdvice`
- Designing and documenting an API with an OpenAPI (OAS 3.1) spec, and testing it with Postman
- Sharing the OpenAPI spec as the contract, and integrating with a frontend team
- Presenting your work and the basics of what deployment involves

9.2 Capstone Submission

7. Push your complete Spring Boot API to GitHub with clean layered packages, a clear README, and a sensible commit history
8. Include an OpenAPI spec at `docs/openapi.yaml` covering every endpoint, plus a Postman collection that tests them including failure cases
9. Ensure CORS allows the frontend, and confirm the full loop works end to end
10. Prepare a 5-minute demo of your API and its integration with the frontend

9.3 Congratulations — Phase 1 Complete

Take a moment to appreciate the distance travelled. In eight weeks you have gone from first principles to a working, database-backed web API that a real frontend consumes. That is a genuine engineering achievement. Be proud, rest well, and get ready for what is next.

Coming Next — Phase 2: Beyond the Foundation

Jim & Anderson will lead you into production-grade backend engineering. Phase 2 builds on this foundation with topics such as security and authentication, automated testing, more advanced Spring, and deploying real services to the cloud. The fundamentals you mastered in Phase 1 are exactly what everything ahead is built on.

Closing Reflection

"For no other foundation can anyone lay than that which is laid."

— 1 Corinthians 3:11 (NKJV)

Everything you build from here will stand on the foundation of these eight weeks — just as every good life is built on a foundation that lasts. You have laid your foundation with diligence. Build the rest with the same care, and remember Who gave you the ability to build at all.

Glossary of Terms

Term	Definition
API Contract	The agreed set of endpoints, methods, and JSON shapes both frontend and backend rely on
Capstone	A culminating project that combines everything learned into one finished product
CORS	Cross-Origin Resource Sharing — the browser rule an API must satisfy to allow calls from another origin
DTO	Data Transfer Object — a record/class describing exactly the data an API sends or receives
Deployment	Putting an application on a server so it is reachable beyond your own machine
Facade	The layer that orchestrates a use case — coordinating services and mappers and owning the transaction
Full-Stack	Involving both the frontend (user interface) and backend (server and data) of an application
Layered Architecture	Separating code into controller → facade → service → repository layers, each with one job
Mapper	A dedicated class that converts between entities (model) and DTOs — one per resource
OpenAPI (OAS)	A standard file (openapi.yaml, v3.1) describing an entire REST API; the design artefact and contract
Origin	The combination of protocol, domain, and port that identifies where a page or API lives
Package-by-Layer	Organising code into packages named for their job: controller, facade, service, repository, ...
README	The Markdown file documenting what a project is and how to run it
Schema (OpenAPI)	A reusable data-shape definition under components/schemas, referenced with \$ref
Validation	Checking incoming data against rules before accepting it, e.g. with @Valid
@RestControllerAdvice	A Spring class that handles exceptions across the whole API in one place